CS126 WAFFLES Coursework Report [2127935]

CustomerStore

Overview

For CustomerStore, the focus was very much on finding a way to store the data in a way that when it would be accessed it would be accessed in a way that would be immediately useful. This was done with a large focus on hashmaps and binary trees. For searching/querying specific IDs, a hashmap would be used for the fast access time it would provide. For sorting the store of data itself it would make use of the binary tree where the data entered into it would be automatically positioned into the tree in the correctly sorted location (dependent on the comparator provided).

This method was less effective in regards to the space complexity, storing more data just in different orders and in differing data structures, however the guide puts much more focus on optimizing the time complexity; and the module is focused on data structures, rather than sorting algorithms.

Space Complexity

Data structures:

- customerHash:
  Hashmap <Long to Customer>
  O(n)
  Space used linearly grows where `n` is total customers added.
  To hash the customerID to the customer object, for fast retrieval in getCustomer().
- customerIDs:
  Binary Search Tree <Customer>
  O(n)
  Space used linearly grows where `n` is total customers added.
  To store and order the customers by their ID, so no sort is needed in getCustomers().
- customerNames:
  Binary Search Tree <Customer>
  O(n)
  Space used linearly grows where `n` is total customers added.
  To store and order the customers by their Names (alphabetically), so no sort is needed in getCustomersByName().
- IDblacklist
  Hashset <Long>
  O(n)
  Space used linearly grows where `k` is total IDs blacklisted.
  To store the IDs that have been blacklisted, so when adding, we can check if the ID is contained in the store quickly.

Hence the final space complexity is O(n+n+n+k) = O(3n+k); as we know n >= k (since we wont number of blacklisted IDs can't exceed the number of customers),

we can then infer space complexity = O(3n) = O(n) as it scales to infinity.

Store	Worst Case	Description
CustomerStore	O(n)	A hashmap, hashset and two binary trees have been used to store all of the data.

Time Complexity

addCustomer(Customer c)

There are several stages of the addition to figure out how long it takes to add to it:

* Checking if the customer is valid in data checker
    - In the data checker it loops 16 times to check something, but this does not change on the customer, hence constant time complexity, O(1).
* Checking if the customer is not contained in the IDblacklist
    - IDblacklist is a hashmap hence has O(1) retrieval time*, hence constant time complexity.
* Checking if the customer is contained in the customerHash
    - customerHash is a hashmap hence has O(1) retrieval time*, hence constant time complexity.
* If it is contained: deleting from stores; adding to IDblacklist
    - Deleting from the hashmaps have an average time complexity of O(1).
    - Deleting from the binary search trees have an time complexity of the height of the tree, which we will denote O(h).
    - Adding to the IDblacklist has an average time complexity of O(1).
* If it is not contained: adding to stores
    - Adding to the hashmaps have an average time complexity of O(1)
    - Adding to the binary searcg trees have an time complexity of the height of the tree, which we will denote O(h)

Hence we can conclude that the average time complexity of adding is O(1+1+...+1+h1+h2) = O(6+h1+h2) = O(h1+h2), where whichever height is bigger (h1 or h2) will be the final height and time complexity of adding to the store.

*Hashmaps have a high probability of O(1) time complexity (in retrieving, adding and deleting), however it relies on the data being distributed evenly throughout the buckets;
hence in its worst case scenario it is O(n), however since we are addressing the average case, we will assume that the data is well distributed.

*The trees are not self balancing meaning that they do not have a time complexity of O(log n); instead they have O(h).

addCustomer(Customer[] c)

This is far more simple, as it simply takes the list of `n` customers, and applies addCustomer(Customer c) to them. As that has time complexity of O(h):

Time complexity is O(h + h + ... h) = O(nh). As `n` >= `h` (in the average case - only in the worst case will the height equal the number of elements), we can conclude it has a time complexity of O(n); but it is more accurate to say O(nh), as h depends on n.

getCustomer(Long id)

Now that they have all been added to the stores, it is far simpler to retrieve the data that we want. As this is retrieving a Customer from a long, we can use the customerHash hashmap to check if the Customer exists, and if it does then retrieve the Customer in O(1) time (on average).

getCustomers()

This is taking the customerIDs binary search tree, and is converting that to an array through an inorder traversal. As the tree has `n` nodes, we can conclude that it has to traverse through all of them, giving us an O(n) time complexity to return the array we want.

getCustomers(Customer[] c)

This is similar, as instead of using the pre-existing customerIDs, we create a tree and manually insert everything, to then turn that into an array.

To insert `n` items into the tree has time complexity O(nh), and then to convert that into an array is O(n). O(n + nh) = O(nh) as `nh` will always be bigger

than `n`, so the time complexity is O(nh).

getCustomersByName()

This is taking the customerNames binary search tree, and is converting that to an array through an inorder traversal. As the tree has `n` nodes, we can conclude that it has to traverse through all of them, giving us an O(n) time complexity to return the array we want.

Note; the method of comparing the strings (the customer names) makes use of a comparator that only requires O(1) time complexity.

getCustomersByName(Customer[] c)

This is similar, as instead of using the pre-existing customerNames, we create a tree and manually insert everything, to then turn that into an array.

To insert `n` items into the tree has time complexity O(nh), and then to convert that into an array is O(n). O(n + nh) = O(nh) as `nh` will always be bigger than `n`, so the time complexity is O(nh).

getCustomersContaining(String s)

By using the convertAccentsFaster, we can use the statically generated hashmap (initialized at the start of its use) and access each of the converted characters in O(1) time. Then by using the length of the string we are converting, we eventually get the string that we are wanting to compare, and since this is not dependent on the size of the accentAndConvertedAccent array, we can say that `a` (the time taken to convert accents) is significantly smaller. Then for the method in which the strings are checked, using the customerNames tree, we traverse inorder, leading to O(n) minimum search time. However, then we add then back to a binary search tree to ensure it is in the correct order, and then convert the final tree. This results it being in order, and leads to a complexity of O(a + n*(a+b) + nh) where `n` is the number of nodes, `a` is the average time to convert accents, `b` is the average string search time, and `h` is the height of customerNames. `nh` > `n*(a+b)`, so we can conclude the time complexity is O(nh), through the recursive actions of the function.

Method	Average Case	Description
addCustomer(Customer c)	O(h)	Linearly scales with h, where `h` is the height of the deepest binary search tree.
addCustomer(Customer[] c)	O(nh)	Where `n` is the number of customers being added and where `h` is the height of the deepest binary search tree.
getCustomer(Long id)	O(1)	Hashmap is used to get the Customer at constant time complexity.
getCustomers()	O(n)	Where `n` is the number of nodes in the tree customerIDs, traversing it inorder.
getCustomers(Customer[] c)	O(nh)	Where `n` is the number of customers in c, and h is the height of the tree made in the function.
getCustomersByName()	O(n)	Where `n` is the number of nodes in the tree customerNames, traversing it inorder.
getCustomersByName(Customer[] c)	O(nh)	Where `n` is the number of customers in c, and h is the height of the tree made in the function.
getCustomersContaining(String s)	O(a+nh*(a+b))	Where `n` is the number of customers in the store, where `h` is the height of the tree made in the function, a is the average time to convert to normalised form, and b is the time taken to see if the string is a substring.

FavouriteStore

Overview

Favourite store was made with the intention of finding ways to store data in a way that would be useful for all of the more interesting functions we were trying to design. For storing the data initially use was needed to be made of hashmaps and binary trees, which gave similar functionality to restaurant store and to customer store. The difference comes in with the additional data structures of more hashmaps that map customerID and restaurantID to a binary tree of associated favourites, and another hashmap mapping them to an integer of the count. This was all done with the design intention of being able to order customer and restaurants individually within the respective favourite cases.

Space Complexity

Data Structures:

- favouriteArray
  ArrayList <Favourite>
  O(n)
  Space used linearly grows where n is the total favourites added.
  This is used to find the favourites which have the same CustomerIDs and same RestaurantIDs.
- favouriteHash
  Hashmap <Long, Favourite>
  O(n)
  Space used linearly grows where n is the total favourites added.
  This is used to check if a favouriteID is contained within the set of data and for getFavourite().
- favouriteIDs
  Binary Search Tree <Favourite>
  O(n)
  Space used linearly grows where n is the total favourites added.
  This is used to order the favourites by their ID on load up so it can be returned instantly by getFavourites().
- favouriteCustomers
  Hashmap <Long, Binary Search Tree <Favourite>>
  O(n*m)
  Space used grows by multiplying the size of the tree (m) with the size of the hashmap (n).
  This is used for getFavouritesByCustomerID() as the tree automatically adds them to the tree in the correct order so it can be immediately returned. Also used for the set functions.
- favouriteRestaurant
  Hashmap <Long, Binary Search Tree <Favourite>>
  O(n*m)
  Space used grows by multiplying the size of the tree (m) with the size of the hashmap (n).
  This is used for getFavouritesByRestaurantID() as the tree automatically adds them to the tree in the correct order so it can be immediately returned. Also used for the set functions.
- customerToCount
  Hashmap <Long, Integer>
  O(n)
  Space used linearly grows where n is the number of customers that have favourited something.
  This is used for the getTopCustomersByFavouriteCount() as this means we can retrieve the number of favourites off of the hashmap instantly for comparison.
- restaurantToCount
  Hashmap <Long, Integer>
  O(n)
  Space used linearly grows where n is the number of restaurants that have been favourited.
  This is used for the getTopRestaurantByFavouriteCount() as this means we can retrieve the number of favourites off of the hashmap instantly for comparison.

  Hence we can conclude that the space complexity is O(n + n + n + … + n + nm + nm) = O(n + nm) = O(nm), where m is the tree with the largest number of nodes.

Store	Worst Case	Description
FavouriteStore	O(nm)	Where n is the number of favourites being added, and m is the size of the largest tree contained in favouriteCustomers or favouriteRestaurants.

Time Complexity

addFavourite(Favourite f)

This also can be broken down into phases.

* Firstly we check if the favourite is valid, which is of O(1) complexity
* Then we check if the blacklist contains the ID which as it is a hashmap it is O(1) complexity
* Then we cycle through the array to find the favourites with customerID and restaurantID being identical. This takes O(n) complexity to cycle through the n elements.
* Then if there is a value is contained within the array with the same IDs, then we check which favourite is the latest.
    - If the current favourite is later then we add it to the blacklist with addToHashTree()
    - If the current favourite is older then we add the container to the blacklist, and update the stores with addToStores() and deleteFromStores().
* If the favouriteID is the same, we then deleteFromStores and put the current favourite into the IDblacklist.
* Otherwise we add to the stores.

We break this down further with the functions of addToHashTree(), deleteFromHashTree(), addToStores() and deleteFromStores().

addToHashTree() updates a HashTree using O(1) get and put time, and O(h) insert time where h is the height. deleteFromHashTree() is very similar.

addToStores() and deleteFromStores() then has the job of wanting to update all of the different stores as described in space complexity. This involves adding to an arraylist – O(1) – adding to a hash map – O(1) – and adding to a binary search tree – O(h) – and finally adding to hashtrees, which is O(h). This is identical for deletion. This culminates in the final time complexity being O(h) of the favouriteIDs.

Hence we can finally sum this all up together, and find that the two most prominent growing values are O(n+h), where n is from cycling through the arraylist, and h is from adding/deleting from a binary search tree (favouriteIDs).

addFavourite(Favourite[] f)

This once more is far more simple, as it is doing the addFavourite repeatedly. It repeats it n times, where k is the number of favourites in f. This results in the complexity of O(n*(n+h)) = O(n^2 + nh).

getFavourite(Long id)

This uses a hashmap of Ids to favourites, using the hashmap of favouriteHash. Once it has check it actually contains favouriteHash, it then returns the value associated with the key of favouriteHash in O(1).

getFavourites()

This converts the binary search tree to an array, which is an inorder traversal, of the tree favouriteIDs. Since it traverses through n nodes, it has a time complexity of O(n).

getFavouritesByCustomerID(Long id)

This is very similar, as all that must be done is to gather the binary tree that we want to output, gotten using the hashmap of Ids to binary trees, called favouriteCustomers, hence that uses O(1) time complexity. Afterwards it converts this to an array, which traverses in order, so it is O(n) time complexity.

getFavouritesByRestaurantID(Long id)

This is very similar, as all that must be done is to gather the binary tree that we want to output, gotten using the hashmap of Ids to binary trees, called favouriteCustomers, hence that uses O(1) time complexity. Afterwards it converts this to an array, which traverses in order, so it is O(n) time complexity.

getCommonFavouriteRestaurants(Long id1, Long id2)

What this function does is that it converts the tree of customers from id2 into a hashmap of RestaurantIds to the favourite through a recursive getHashMap() method. Then it recursively traverses through the tree of customers from id1 to see what RestaurantIds are contained in the newly constructed hashmap. This means first making the hashmap takes O(m) complexity as we traverse through m elements inorder. Then we traverse through n elements of the tree so we get O(n). Finally there is converting the subsequent tree into an array, where this final size of the last tree is k. Since checking if it is contained is O(1) due to hashmap operations, we can finally find that it has a time complexity of O(n+m+k), where n is the size of the tree from id1, and m is the size of the tree from id2 and k is the number of shared elements.

getMissingFavouriteRestaurants(Long id1, Long id2)

What this function does is that it converts the tree of customers from id2 into a hashmap of RestaurantIds to the favourite through a recursive getHashMap() method. Then it recursively traverses through the tree of customers from id1 to see what RestaurantIds are contained in the newly constructed hashmap. This means first making the hashmap takes O(m) complexity as we traverse through m elements inorder. Then we traverse through n elements of the tree so we get O(n). Finally there is converting the subsequent tree into an array, where this final size of the last tree is k. Since checking if it is not contained is O(1) due to hashmap operations, we can finally find that it has a time complexity of O(n+m+k), where n is the size of the tree from id1, and m is the size of the tree from id2 and k is the number of not shared elements.

getNotCommonFavouriteRestaurants(Long id1, Long id2)

This was a set symmetric difference operator, hence we can use some knowledge that set symmetric difference of sets A and B is equal to the set difference of the union of A and B and the intersection of A and B. Since we have developed a set difference function in the getMissingFavouriteRestaurant() and we get the intersection in getCommonFavouriteRestaurants(), we can just get the union with getAll_Recursive on both the trees gotten from id1 and id2; resulting in O(n+m). Then we get the intersection which is O(n+m+k) and then we find the set difference of the two, which is O(n+m). Finally we convert this to an array of size l, so we can conclude the time complexity is O(n+m+n+m+k+n+m+l) = O(3n+3m+k+l) = O(n+m+k+l).

getTopCustomersByFavouriteCount()

This method makes use of the entries in favouriteCustomers to go through them each and insert each key into a tree, along with the associated count gathered from customersToCount. The tree automatically collects them, so then we can gather the first 20 are the values that we want. This means we loop through O(h) insert operations n times, so we get O(nh) and then convert to an array which is another n operations, finally an additional 20 to gather the last entries. The final complexity hence is O(nh + n + 20) = O(nh), where n is the size of favouriteCustomers, and h is the height of the tree.

getTopRestaurantsByFavouriteCount()

This method makes use of the entries in favouriteRestaurants to go through them each and insert each key into a tree, along with the associated count gathered from restaurantsToCount. The tree automatically collects them, so then we can gather the first 20 are the values that we want. This means we loop through O(h) insert operations n times, so we get O(nh) and then convert to an array which is another n operations, finally an additional 20 to gather the last entries. The final complexity hence is O(nh + n + 20) = O(nh), where n is the size of favouriteRestaurant, and h is the height of the tree.

Method	**Average Case**	**Description**
addFavourite(Favourite f)	O(n+h)	Where n is the number of elements contained in the arraylist, and h is the height of the binary search tree favouriteIDs.
addFavourite(Favourite[] f)	O(n^2 + nh)	Where n is the number of elements in f and h is the height of the binary search tree.
getFavourite(Long id)	O(1)	Uses a hashmap, hence it is constant time to retrieve the favourite.
getFavourites()	O(n)	Where n is the number of elements in the tree favouriteIDs, already ordered.
getFavouritesByCustomerID(Long id)	O(n)	Where n is the number of elements in the tree favouriteCustomers,get(id).
getFavouritesByRestaurantID(Long id)	O(m)	Where m is the number of elements in the tree favouriteRestaurants.get(id).
getCommonFavouriteRestaurants(Long id1, Long id2)	O(n+m+k)	Where n is the number of elements in the tree favouriteRestaurants.get(id1), where m is the number of elements in the tree favouriteRestaurants.get(id2)

Method	Average Case	Description
		and where k is the number of elements they have in common.
getMissingFavouriteRestaurants(Long id1, Long id2)	O(n+m+k)	Where n is the number of elements in the tree favouriteRestaurants.get(id1), where m is the number of elements in the tree favouriteRestaurants.get(id2) and where k is the number of elements they don't have in common.
getNotCommonFavouriteRestaurants(Long id1, Long id2)	O(n+m+k+l)	Where n is the number of elements in the tree favouriteRestaurants.get(id1), where m is the number of elements in the tree favouriteRestaurants.get(id2), where k is the number of elements they don't have in common, and l is the size of array being returned.
getTopCustomersByFavouriteCount()	O(nh)	Where n is the number of elements in favouriteCustomers, and h is the height of the tree formed.
getTopRestaurantsByFavouriteCount()	O(nh)	Where n is the number of elements in favouriteRestaurants, and h is the height of the tree formed.

RestaurantStore

Overview

Restaurant store was very much a natural extension of Customer store, in which the main logical area was structured around adding to a series of binary trees that ordered in a logical fashion using comparators. There were five binary trees used for all of the different search categories, as well as some hashmaps for utility features, however for the most part it really was a simple follow up from the materials in Customer store.

Space Complexity

Data Structures:

- restaurantHash
  Hashmap <Long, Restaurant>
  O(n)
  Space used linearly grows where n is the total restaurants added.
  This is used for checking if an ID is contained in the stores, and returning the restaurant quickly for getRestaurant.

- restaurantIDs
  Binary Search Tree <Restaurant>
  O(n)
  Space used linearly grows where n is the total restaurants added.
  This is used to order the restaurants in ascending order of ID, used to immediately retrieve the data in getRestaurants()
- restaurantNames
  Binary Search Tree <Restaurant>
  O(n)
  Space used linearly grows where n is the total restaurants added.
  This is used to order the restaurants in alphabetical order by name, used to immediately retrieve the data in getRestaurantsByName()
- restaurantDates
  Binary Search Tree <Restaurant>
  O(n)
  Space used linearly grows where n is the total restaurants added.
  This is used to order the restaurants by the most recent restaurants established, used to immediately retrieve the data in getRestaurantsByDateEstablished()
- restaurantStars
  Binary Search Tree <Restaurant>
  O(n)
  Space used linearly grows where n is the total restaurants added.
  This is used to order the restaurants in order of number of Warwick stars, used to immediately retrieve the data in getRestaurantsByWarwickStars()
- restaurantRatings
  Binary Search Tree <Restaurant>
  O(n)
  Space used linearly grows where n is the total restaurants added.
  This is used to order the restaurants in order of customer ratings, used to immediately retrieve the data in getRestaurantsByRating()
- IDblacklist
  Hashset <Long>
  O(k)
  Space grows linearly where k is the restaurants blacklisted.
  This is used to find the data quickly, to see if the ID is contained in the blacklist.

By combining all of this together we find it is O(6n + k) = O(n) for the total space complexity.

Store	**Worst Case**	**Description**
RestaurantStore	O(n)	Where five binary search trees have been used, a hashset and a hashmap, increasing linearly.

Time Complexity

addRestaurant(Restaurant r)

This function is similar in that first we check whether the restaurant is valid, which is an O(1) operation, then we check if the IDblacklist contains the ID of r, which is also O(1) as it is a hashmap. Following this we check whether the stores contain r using the hashmap, and if they do then we delete from all of the stores. Alternatively, if it is not contained, we then add to the stores. Both adding and deleting from each binary search tree is on average O(h) where h is the height of the trees. Hence we can then

conclude that the time complexity of this is O(h) where h is the height of the deepest tree, as the trees are unbalanced.

addRestaurant(Restaurant[] r)

Once again we do a similar tactic, to previous stores, where we add items n times to the stores. This calls the addRestaurant() function n times, leading to a time complexity of O(nh).

getRestaurant(Long id)

This uses similar methods to other stores, where we use the restaurantHash to check if the id is contained in the set of data, and if it is then retrieved using O(1) time complexity.

getRestaurants()

We use the inbuilt tree organised by an ID comparator and convert it into an array, traversing n nodes, resulting in a time complexity of O(n).

getRestaurants(Restaurants[] r)

We add the number of elements in the entered array to a tree, with the insertion time for n elements being O(hn), then we convert it to an array which is O(n), resulting in an overall time complexity of O(nh).

getRestaurantsByName()

We use the inbuilt tree organised by a Name comparator and convert it into an array, traversing n nodes, resulting in a time complexity of O(n).

getRestaurantsByDateEstablished()

We use the inbuilt tree organised by an date comparator and convert it into an array, traversing n nodes, resulting in a time complexity of O(n).

getRestaurantsByDateEstablished(Restaurants[] r)

We add the number of elements in the entered array to a tree, with the insertion time for n elements being O(hn), then we convert it to an array which is O(n), resulting in an overall time complexity of O(nh).

getRestaurantsByWarwickStars()

We use the inbuilt tree organised by a Warwick stars comparator (as well as excluding any that don't have any stars) and convert it into an array, traversing n nodes, resulting in a time complexity of O(n).

getRestaurantsByRating(Restaurants[] r)

We add the number of elements in the entered array to a tree, with the insertion time for n elements being O(hn), then we convert it to an array which is O(n), resulting in an overall time complexity of O(nh).

getRestaurantsByDistanceFrom(float latitude, float longitude)

We create a new tree that adds in a new RestaurantDistance object whilst iterating through an inorder restaurantIDs array. As inserting takes O(h), and it does n times, and then we convert the tree into an array using an inorder traversal of n elements, we can conclude it has a time complexity of O(nh).

getRestaurantsByDistanceFrom(Restaurant[] float latitude, float longitude)

We create a new tree that adds in a new RestaurantDistance object whilst iterating through the array given. We then add to a tree that automatically sorts using a comparator. As inserting takes O(h), and it does n times, and then we convert the tree into an array using an inorder traversal of n elements, we can conclude it has a time complexity of O(nh).

getRestraurantsContaining(String searchTerm)

Here we break down the function into checking if it has a name match, cuisine match or a place match. This makes use of three contains, which in terms of the search string time we will call O(b). Then we create a temporary tree, which then is filled recursively in an inorder traversal going through all n nodes, with the conversion of accents only happening twice at the start. This results in a search time of(n*b + nh) as we insert to the tree repeatedly, finally converting to the array, so we can conclude the final complexity is O(n*(b+h)).

Method	**Average Case**	**Description**
addRestaurant(Restaurant r)	O(h)	Where h is the height of the deepest tree of all of the stores.
addRestaurant(Restaurant[] r)	O(nh)	Where n is the number of items being added, and h is the height of the deepest tree in all of the stores.
getRestaurant(Long id)	O(1)	Constant time complexity from using hashmap retrieval.
getRestaurants()	O(n)	Where n is the number of elements in the restaurant stores.
getRestaurants(Restaurant[] r)	O(nh)	Where n is the number of elements in r, and h is the height of the tree made in the function.
getRestaurantsByName()	O(n)	Where n is the number of elements in the restaurant stores.
getRestaurantsByDateEstablished()	O(n)	Where n is the number of elements in the restaurant stores.
getRestaurantsByDateEstablished(Restaurant[] r)	O(nh)	Where n is the number of elements in r, and h is the height of the tree made in the function.
getRestaurantsByWarwickStars()	O(n)	Where n is the number of elements in the restaurant stores.

Method	Average Case	Description
getRestaurantsByRating(Restaurant[] r)	O(nh)	Where n is the number of elements in r, and h is the height of the tree made in the function.
getRestaurantsByDistanceFrom(float lat, float lon)	O(nh)	Where n is the number of elements in r, and h is the height of the tree made in the function.
getRestaurantsByDistanceFrom(Restaurant[] r, float lat, float lon)	O(nh)	Where n is the number of elements in r, and h is the height of the tree made in the function.
getRestaurantsContaining(String s)	O(nb+nh)	Where n is the number of elements in the stores, b is the time taken to check if a string is contained, and h is the height of the tree created in the function.

Util

Overview

- **ConvertToPlace**
  - This incredibly lazy solution goes through the array of places linearly until it finds the latitude and longitude that are equal, going through an array of places already defined.
- **DataChecker**
  - This is a large collection of functions that check a large number of parameters, with most checks just making sure that attributes are not null.
- **HaversineDistanceCalculator (HaversineDC)**
  - This defines a subsidiary function that converts the floats to radians, and finds the distance between two locations. Finally, then we use a round function which rounds it to the correct digits of one decimal place in each of the main functions.
- **KeywordChecker**
  - -
- **StringFormatter**
  - This works by assigning the accentAndConvertedAccent to a hashmap, and then in the string converter function it assigns the characters much faster by automatically getting the character instantly.

Space Complexity

Util	Worst Case	Description
ConvertToPlace	O(n)	Where n is the number of places contained in the array made statically.
DataChecker	O(1)	There is nothing stored of increasing size.
HaversineDC	O(1)	There is nothing stored of increasing size.
KeywordChecker	-	-
StringFormatter	O(n)	Where n is the number of characters contained in the accentAndConvertedAccent.

Time Complexity

Util	Method	Average Case	Description
ConvertToPlace	convert(float lat, float lon)	O(n)	Linearly searches through all of the places.
DataChecker	extractTrueID(String[] repeatedID)	O(1)	No repeated comparisons need to be made.
DataChecker	isValid(Long id)	O(1)	Only needs to loop 16 times which doesn't change overtime.
DataChecker	isValid(Customer customer)	O(1)	Does not loop so stays the same.
DataChecker	isValid(Favourite favourite)	O(1)	Does not loop so stays the same.
DataChecker	isValid(Restaurant restaurant)	O(1)	Does not loop so stays the same.
DataChecker	isValid(Review review)	-	-
HaversineDC	inKilometres(float lat1, float lon1, float lat2, float lon2)	O(1)	Performs simple calculation that does not change.

Util	Method	Average Case	Description
HaversineDC	inMiles(float lat1, float lon1, float lat2, float lon2)	O(1)	Performs simple calculation that does not change.
KeywordChecker	isAKeyword(String s)	-	-
StringFormatter	convertAccentsFaster(String s)	O(1)	Loops through the size of string, which doesn't grow.

Reference list

Andhariya, A. (2017). *How is HashSet implemented internally in Java?* [online] Code Pumpkin. Available at: https://codepumpkin.com/hashset-internal-implementation/ [Accessed 6 Mar. 2022].

baeldung (2018). *Merge Sort in Java | Baeldung*. [online] www.baeldung.com. Available at: https://www.baeldung.com/java-merge-sort [Accessed 25 Feb. 2022].
baeldung (2019a). *Comparator and Comparable in Java | Baeldung*. [online] Baeldung. Available at: https://www.baeldung.com/java-comparator-comparable [Accessed 25 Feb. 2022].

baeldung (2019b). *Get Substring from String in Java | Baeldung*. [online] www.baeldung.com. Available at: https://www.baeldung.com/java-substring [Accessed 5 Mar. 2022].

baeldung (2020). *Creating a Generic Array in Java | Baeldung*. [online] www.baeldung.com. Available at: https://www.baeldung.com/java-generic-array [Accessed 25 Feb. 2022].
Delgado, A.M.M. (2020). *Comparing Long Values in Java | Baeldung*. [online] www.baeldung.com. Available at: https://www.baeldung.com/java-compare-long-values [Accessed 23 Feb. 2022].

Devcubicle (2019). *Pass 2d Array to Method in Java*. [online] DevCubicle. Available at: https://www.devcubicle.com/pass-2d-array-to-method-in-java/ [Accessed 5 Mar. 2022].

differencebetween. (n.d.). *Difference Between Union and Intersection | Difference Between*. [online] Available at: http://www.differencebetween.net/language/words-language/difference-between-union-and-intersection/ [Accessed 21 Mar. 2022].

Educative: Interactive Courses for Software Developers. (n.d.). *What is a Binary Search Tree?* [online] Available at: https://www.educative.io/edpresso/what-is-a-binary-search-tree [Accessed 21 Mar. 2022].

Fernandez-Baca, D. and Kautz, S. (n.d.). *Sorting and Generic Methods Based on the notes from*. [online] Available at:

https://cs.brynmawr.edu/Courses/cs206/fall2013/slides/08_Sorting_GenericMethod.pdf [Accessed 25 Feb. 2022].

GeeksForGeeks (2016). *Date class in Java (With Examples)*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/date-class-java-examples/ [Accessed 20 Feb. 2022].

GeeksForGeeks (2019). *PrintStream println(boolean) method in Java with Examples*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/printstream-println boolean-method-in-java-with-examples/#:~:text=The%20println(boolean)%20method%20of [Accessed 20 Feb. 2022].

GeeksForGeeks (2021). *Modulo or Remainder Operator in Java*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/modulo-or-remainder-operator-in-java/#:~:text=Modulo%20or%20Remainder%20Operator%20returns [Accessed 20 Feb. 2022].

GeeksforGeeks (2014). *QuickSort - GeeksforGeeks*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/quick-sort/ [Accessed 23 Feb. 2022].

GeeksforGeeks (2016a). *Arrays.sort() in Java with examples*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/arrays-sort-in-java-with-examples/ [Accessed 23 Feb. 2022].

GeeksforGeeks (2016b). *Collections.sort() in Java with Examples*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/collections-sort-java-examples/ [Accessed 25 Feb. 2022].

GeeksforGeeks (2016c). *Comparator Interface in Java with Examples*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/comparator-interface-java/ [Accessed 23 Feb. 2022].

GeeksforGeeks (2018a). *How compare() method works in Java*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/how-compare-method-works-in-java/ [Accessed 23 Feb. 2022].

GeeksforGeeks (2018b). *Java long compareTo() with examples*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/java-long-compareto-with-examples/ [Accessed 23 Feb. 2022].

Goodrich, M.T. and Tamassia, R. (2015). *Data structures and algorithms in Java*. [online] New York: John Wiley. Available at: http://bedford-computing.co.uk/learning/wp-content/uploads/2016/08/Data-Structures-and-Algorithms-in-Java-6th-Edition.pdf [Accessed 5 Mar. 2022].

Irfan, M. (2021). *Initialize All Array Elements to Zero in Java*. [online] Delft Stack. Available at: https://www.delftstack.com/howto/java/java-initialize-array-elements-to-zero/ [Accessed 5 Mar. 2022].

Jaiswal, A. (n.d.). *Symmetric Difference | Brilliant Math & Science Wiki*. [online] brilliant.org. Available at: https://brilliant.org/wiki/sets-symmetric-difference/#:~:text=The%20symmetric%20difference%20of%20set [Accessed 21 Mar. 2022].
JavaTpoint (n.d.). *Java SortedSet - Javatpoint*. [online] www.javatpoint.com. Available at: https://www.javatpoint.com/java-sortedset#:~:text=Java%20SortedSet%20interface [Accessed 21 Mar. 2022].

Programiz (n.d.). *Java Program to Check Whether a Character is Alphabet or Not*. [online] www.programiz.com. Available at: https://www.programiz.com/java-programming/examples/alphabet [Accessed 5 Mar. 2022].

Rahman, M. (2021). *HashMap Implementation for Java*. [online] The Startup. Available at: https://medium.com/swlh/hashmap-implementation-for-java-90a5f58d4a5b [Accessed 6 Mar. 2022].

Rai, Y. (2018). *Java HashMap Implementation in a Nutshell - DZone Java*. [online] dzone.com. Available at: https://dzone.com/articles/custom-hashmap-implementation-in-java [Accessed 5 Mar. 2022].

Singh, C. (2013). *Java String compareTo() Method with examples*. [online] beginnersbook.com. Available at: https://beginnersbook.com/2013/12/java-string-compareto-method-example/ [Accessed 25 Feb. 2022].

Software Testing Help (2020). *Binary Search Tree In Java - Implementation & Code Examples*. [online] www.softwaretestinghelp.com. Available at: https://www.softwaretestinghelp.com/binary-search-tree-in-java/ [Accessed 8 Mar. 2022].
tutorialspoint.com (2019). *Java How to Use Comparator?* [online] www.tutorialspoint.com. Available at: https://www.tutorialspoint.com/java/java_using_comparator.htm [Accessed 25 Feb. 2022].

W3Schools (2019). *Java Arrays*. [online] W3schools.com. Available at: https://www.w3schools.com/java/java_arrays.asp.
W3schools (2019). *Java HashMap*. [online] W3schools.com. Available at: https://www.w3schools.com/java/java_hashmap.asp [Accessed 5 Mar. 2022].
W3schools (2020). *Java For Loop*. [online] W3schools.com. Available at: https://www.w3schools.com/java/java_for_loop.asp [Accessed 5 Mar. 2022].
www.tutorialspoint.com. (n.d.). *Java - Generics*. [online] Available at: https://www.tutorialspoint.com/java/java_generics.htm#:~:text=Generic%20Methods [Accessed 23 Feb. 2022].

www.tutorialspoint.com. (n.d.). *Online Java Compiler - Online Java Editor - Online Java IDE - Java Coding Online - Practice Java Online - Execute Java Online - Compile Java Online - Run*

Java Online. [online] Available at: https://www.tutorialspoint.com/compile_java_online.php [Accessed 5 Mar. 2022].

Yadav, R. (2020). *Get the Separate Digits of an Int Number in Java*. [online] Delft Stack. Available at: https://www.delftstack.com/howto/java/how-to-get-the-separate-digits-of-an-int-number/ [Accessed 20 Feb. 2022].

Yadav, R. (2021). *Compare Strings Alphabetically in Java*. [online] Delft Stack. Available at: https://www.delftstack.com/howto/java/java-compare-strings-alphabetically/ [Accessed 25 Feb. 2022].